

FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

FAST School of Computing · Karachi Campus
Department of Artificial Intelligence & Data Science

AL-2001 · PROGRAMMING FOR ARTIFICIAL INTELLIGENCE

Laboratory · Fall 2026

PROJECT PROPOSAL

ALPHA LMS

Learning Management System · Programming for AI

	Student Name	Roll Number	Section
Member 1	Ashbar Hussain	25k-0028	BAI-3A
Member 2	Muhammad Mudassir	25k-0153	BAI-3A

Course	AL-2001 — Programming for Artificial Intelligence (Laboratory)
Programme / Semester	BS Artificial Intelligence · 3rd Semester
Session	Fall 2026
Lab Instructor	Miss Khadeeja Ashraf
Section	BAI-3A
GitHub Repository	https://github.com/AshbarHussain25k0028/PAI
Date	19 October 2026

Table of Contents

This proposal describes ALPHA LMS before development begins. We have studied through Lab 09. Topics after week 9 are yet to be studied, and we will try to incorporate suitable later concepts as we learn them.

Chapter	Title	Page
	Declaration and Contribution Statement	2
1	Introduction	3
2	Problem Statement and Requirements	3
3	System Design and Architecture	4
4	Implementation	4
5	Dataset	5
6	Results and Analysis	5
7	Testing, Error Handling and Ethics	5
8	Individual Contributions and Conclusion	6
	Appendix A — Setup and Execution	6

Declaration and Contribution Statement

We propose developing ALPHA LMS for our Programming for AI course. Coding has not started. We will record each member’s work during development and acknowledge external libraries, datasets, APIs and any AI assistance. The following descriptions explain planned features and expected outcomes; they do not claim completed implementation or test results.

1. Introduction

1.1 Overview

ALPHA LMS will be a Python console-based Learning Management System for a small school or college class. It will bring course materials, assignments, quizzes and progress records into one place. Teachers will share notes, set deadlines, create quizzes and enter marks. Students will view their courses, submit work, attempt quizzes and check feedback. Rule-based revision suggestions will identify topics to revisit using quiz mistakes. The core project will use resources already available, such as teacher notes, questions, class lists and assignment files. It will work without extra hardware or paid AI access. We have studied through Lab 09; later machine learning and text-processing features will be attempted as the corresponding lessons are covered.

1.2 Objectives and Scope

Our objectives will be to organize learning resources, manage assignments and submissions, support quizzes and feedback, and display progress with useful revision suggestions. The initial version will serve one class or department through local console menus and file storage. It will include separate student, teacher and administrator roles. Live video classes, fees, a public web portal and institution-wide deployment will remain outside the initial scope.

2. Problem Statement and Requirements

Students often receive notes, deadlines and feedback through several channels, making it easy to miss a file or overlook an assignment. Teachers may keep submissions and marks in separate folders or spreadsheets. We aim to provide one organized workflow for these routine tasks. A student will be able to see course materials and upcoming work, while a teacher will be able to review submissions and record feedback. Progress summaries will show completed and missing work without requiring repeated manual comparisons.

3. System Design and Architecture

3.1 Overall Architecture

Role-specific menus will pass requests to course, assignment and quiz components. The application will check role permissions and enrollment before displaying or changing records. Local storage will preserve accounts, materials, submissions and results. Quiz attempts will produce topic-level summaries for rule-based revision suggestions. NumPy and Pandas will prepare analytics, and Matplotlib and Seaborn will display them. Optional API and later text-classification components will remain separate so that the basic LMS can work independently.

3.2 Class Design

A User base class will hold shared account details, with Student and Teacher subclasses providing role-specific behaviour. Course, Assignment and Quiz classes will organize teaching activities. Submission and attempt records will connect each student to the correct course task. Administrator menus will manage accounts and enrollment.

Constructors, attributes and methods will manage object state. Encapsulation will protect marks and submission changes. Inheritance and overriding will support student and teacher roles, while polymorphism will let the system call a common account interface.

3.3 Choice of Data Structures

Lists will preserve materials and questions in display order. Dictionaries will link IDs to courses, accounts and tasks, avoiding repeated scans. Sets will support unique enrollments, and tuples will group fixed identifiers. NumPy arrays and Pandas Series/DataFrames will support numerical and labelled progress analysis.

4. Implementation

4.1 Module Overview

We will separate account roles, course resources, assignments, quizzes, storage, recommendations and analysis. An optional public question-bank API may supply supplementary practice questions, subject to teacher review and local caching. Teacher-created questions will remain primary, and internet access will not be required for the core LMS.

4.2 Lab Coverage — Where Each Concept Is Used

We have completed Labs 01–09. Variables, operators, type conversion, conditions, loops, break/continue and comprehensions will support menus and record filtering. Functions will use parameters, defaults, returns and scope, with zip, enumerate, map, filter and lambda where useful. OOP will organize roles and teaching activities. TXT/CSV/JSON files, exceptions, requests and response parsing will support storage and optional API access. NumPy creation, indexing, slicing, reshaping, broadcasting and vectorization will support calculations and loop comparisons. Pandas will support loading, missing values, selection, transformation, groupby and merging; Matplotlib and Seaborn will support charts.

4.3 Key Implementation Details

Teachers will register materials and assignments under a course ID. Students will submit work against an assignment ID, and the system will retain submission times and status. Multiple-choice quizzes will use teacher-approved answer keys; teachers will mark assignments and control final marks. Revision rules will use missed question topics to suggest relevant notes. These suggestions will guide practice rather than label a student's ability.

Topics after week 9 are yet to be studied. We will try week 10/Lab 10 model fitting and train/test evaluation for notes tagging, followed by week 12/Lab 11 comparison and cross-validation. Week 13/Lab 12 cleaning, tokenization, normalization, Bag-of-Words and TF-IDF may support notes search and classification. Week 14/Lab 13 may support optional scanned-note resizing, colour conversion and normalization; this preprocessing will not read handwriting or extract text. Week 15/Lab 14 will guide modularity, documentation, privacy and bias review. The week 11 progress review will help assess feasibility. These are planned attempts, not guaranteed features.

5. Dataset

We will begin with fictional accounts and enrollment records, teacher-created questions, demonstration assignments and notes we are permitted to use. Records will include course, material, assignment and user IDs; deadlines and submission times; quiz answers, scores and topic tags. TXT notes will support later search, while other documents may be shared as files. No dataset has been collected yet.

Later subject tagging will require a varied set of notes labelled by course or topic. We will document the source, size and permissions, avoid exposing real student records, and only attempt classification if sufficient suitable material and time are available.

6. Results and Analysis

6.1 Program Output

We expect separate menus to show courses, resources, deadlines, submissions, quiz results and feedback according to role. Teachers will review class work, while students will see their own progress and revision suggestions. No application output or screenshots are available yet.

6.2 Visualisations

We plan a line chart of class quiz averages over time, a bar chart of assignment completion, a histogram of score distributions and a scatter plot of practice attempts versus scores. Seaborn may show topic-wise performance. Missing work will be identified separately from earned marks, and teacher-only analytics will protect individual results.

6.3 Model Results

We have not studied model training yet. After the relevant lessons, we will try comparing a notes subject classifier with a keyword baseline on the same held-out material. We may report accuracy, category precision/recall and a confusion matrix, with cross-validation if feasible. Vectorization will be fitted on training data only. Notes search will be assessed by checking whether relevant passages appear among the retrieved results. No measured scores are claimed.

7. Testing, Error Handling and Ethics

7.1 Test Cases

We will test enrollment, role permissions, duplicate IDs, missing files, invalid dates, submission ownership, score limits and incomplete quiz attempts. We will check that students cannot change marks or view another student's work. Optional API tests will cover invalid JSON and timeouts. Later search and tagging tests will check relevance, incorrect labels and correction paths.

7.2 Exception Handling

Exception handling will cover invalid choices, malformed records, file failures and unavailable network services. Clear messages will allow corrections without losing valid data. Failed writes will preserve saved records. If an API or experimental AI feature fails, local course materials, quizzes and manual workflows will remain available.

7.3 Ethical Considerations

We will use fictional demonstration accounts and keep real student records out of the public repository. Users will only access records permitted by their role and enrollment. Teachers will control final assessment, and automated topic suggestions will remain explainable and correctable. Notes classification will not be used to judge a student's ability. We are yet to study Ethical AI in week 15 and will try to strengthen privacy, bias and documentation checks after that lecture.

8. Individual Contributions and Conclusion

Both members are in BAI-3A. We will agree on responsibilities before implementation and document each member's work through Git commits and project notes. Development, testing and the final explanation will be shared, while specific contribution shares will reflect the work actually completed.

8.1 Challenges Faced

We expect challenges with connecting records correctly, separating role permissions, managing submission changes and finding enough labelled notes. We will start with one class, stable IDs and simple workflows. Later AI features will remain optional so that we can prioritize a reliable LMS we can explain and test.

8.2 Conclusion and Future Work

ALPHA LMS will organize course materials, assignments, quizzes and progress in one manageable Python project. The first version will apply the material studied through Lab 09 and use rule-based revision suggestions. Topics after week 9 are still to be covered, and we will try to incorporate suitable machine learning, text-processing, optional image-preprocessing and professional-practice concepts as we learn them. We will build and test the core workflow first, then report only the extensions and results actually achieved.

Appendix A — Setup and Execution

We plan to use Python 3, a virtual environment and Jupyter Notebooks for analysis. Initial libraries will include NumPy, Pandas, Matplotlib and Seaborn, with requests for optional API access and standard modules for files and dates. Scikit-learn and OpenCV may be added after their lessons if the corresponding extensions are developed. Verified run instructions will be written after implementation.

The demonstration will use local folders for materials and submissions, safe fictional accounts, sample CSV/JSON records and teacher-approved questions. Any optional API settings will stay separate from source code. A README will explain the actual entry point, supported files, role setup and optional features.

End of Project Proposal